

Scripts d'extractions et de traitement de données de vie réelle d'ARIA à des fins de recherche clinique

ARIA ODM Builder
Notice visuelle du dépôt GitHub



Comment lire ce document

But du PDF

Ce document est conçu pour être lu directement depuis le dossier docs/ du dépôt GitHub.

- ▶ comprendre l'ordre réel du pipeline ;
- ▶ identifier les fichiers importants ;
- ▶ savoir quel script sert à quoi ;
- ▶ comprendre le résultat attendu.

Public visé

Utilisateur non spécialiste du code :
médecin, physicien médical, interne,
chercheur ou collaborateur projet.

Ce PDF fonctionne comme un README
visuel.

Objectif du dépôt

Transformer des extractions ARIA en fichiers patients structurés, contrôlés et exploitables pour la recherche clinique.

Extraire

Récupérer les informations utiles avec les requêtes SQL.

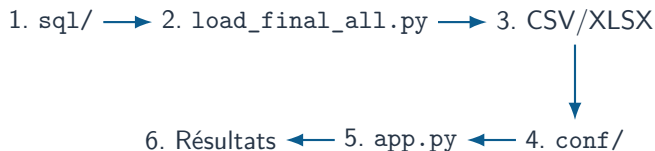
Préparer

Générer les fichiers d'entrée CSV et Excel avec le script principal.

Exporter

Produire un résultat structuré avec l'application Streamlit.

Chaîne complète du projet



- ▶ sql/ contient les requêtes d'extraction.
- ▶ scripts/load_final_all.py lance les requêtes et génère les fichiers d'entrée.
- ▶ app.py charge, contrôle et exporte les données via Streamlit.

Organisation visible sur GitHub

```
ARIA_ODM_Builder/  
|-- conf/  
|-- docs/  
|-- pictures/  
|-- samples/  
|-- scripts/  
|-- sql/  
|-- utils/  
|-- README.md  
|-- app.py  
|-- requirements.txt  
'-- requirements_sql.txt
```

Lecture rapide

Le dépôt suit la chaîne : extraction SQL, préparation par scripts, fichiers d'entrée, configuration, application Streamlit et export.

Point d'entrée utilisateur

Pour l'utilisation guidée, l'utilisateur lance `app.py`. Pour la préparation amont, le script principal est dans `scripts/`.

Étape 1 : requêtes SQL dans sql/

```
sql/  
|-- README.md  
|-- requete_traitement.sql  
|-- requete_formulaire.sql  
'-- requete_complementaire.sql
```

Rôle

Produire les premières extractions nécessaires au pipeline.

- ▶ extraction des données de traitement ;
- ▶ extraction des formulaires cliniques ;
- ▶ extraction d'une source complémentaire si nécessaire.

Guide détaillé : docs/guide_execution_sql.md

Connexion à la base ARIA : où renseigner les paramètres

Où modifier ?

Les informations de connexion sont à renseigner dans le script principal de préparation :

`scripts/load_final_all.py`

fonction : `connecter_bdd()`

chercher : `pyodbc.connect(...)`

- ▶ nom du serveur ;
- ▶ nom de la base ;
- ▶ mode d'authentification ;
- ▶ port si nécessaire.

Exemple à adapter

```
pyodbc.connect(  
    "Driver={ODBC Driver 17 for SQL Server};"  
    "Server=REEMPLIR_ICI_NOM_SERVEUR,PORT;"  
    "Database=REEMPLIR_ICI_NOM_BASE;"  
    "Trusted_Connection=yes;"  
    "TrustServerCertificate=yes;"  
)
```

À retenir : l'utilisateur complète ce bloc avec les paramètres fournis par son environnement de travail.

Étape 2 : préparation dans scripts/

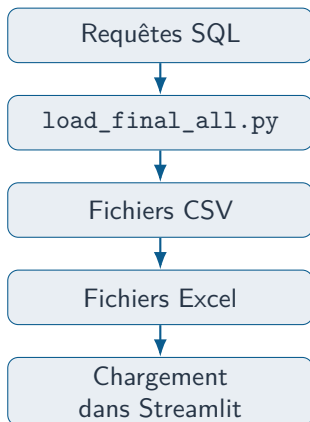
```
scripts/  
|-- README.md  
'-- load_final_all.py
```

Script principal

load_final_all.py centralise l'extraction amont.

Il exécute les requêtes SQL, récupère les résultats et génère les fichiers d'entrée au format CSV et Excel.

Sortie de la préparation amont



Fichiers obtenus

Le script produit les fichiers nécessaires au workflow Streamlit dans deux formats : .csv et .xlsx.

Intérêt

Cette étape évite de mélanger extraction, génération des fichiers d'entrée et utilisation interactive.

Étape 3 : fichiers d'entrée pour Streamlit

Cas réel : fichiers générés

```
traitement_patient.csv / .xlsx  
formulaire_patient.csv / .xlsx  
ethos_patient.csv / .xlsx
```

Cas démo : fichiers fictifs

```
samples/  
|-- csv/  
'-- xlsx/
```

Les fichiers de démonstration permettent de tester l'application sans extraction SQL.

Ces fichiers sont ceux que l'utilisateur charge ensuite dans l'interface Streamlit.

Étape 4 : configuration dans conf/

`mapping.csv`

Indique quelle colonne source correspond à quelle variable utilisée dans le pipeline.

Exemples : identifiant patient, date, dose, diagnostic, fractionnement.

`profiles/*.json`

Décrit un scénario d'extraction :
pathologie, critères, variables attendues et règles de sélection.

Exemples : codes CIM10, variables cliniques, règles de cohorte.

Le mapping dit où lire. Le profil dit quoi sélectionner.

Installer le projet

Application Streamlit

```
python -m venv .venv  
.venv\Scripts\activate  
pip install -r requirements.txt
```

requirements.txt contient les dépendances de l'application.

Extraction SQL

```
python -m venv .venv_sql  
.venv_sql\Scripts\activate  
pip install -r requirements_sql.txt
```

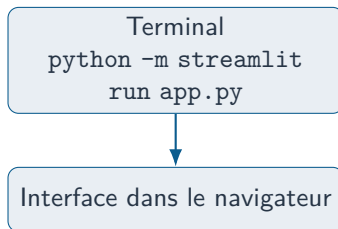
requirements_sql.txt contient les dépendances minimales pour l'extraction SQL.

Étape 5 : lancer l'application Streamlit

Commande

```
python -m streamlit cache clear  
python -m streamlit run app.py
```

- ▶ l'application s'ouvre dans le navigateur ;
- ▶ l'utilisateur charge les fichiers préparés ;
- ▶ le pipeline peut être recalculé depuis l'interface.



Ce que fait app.py



- ▶ interface principale du dépôt ;
- ▶ guide l'utilisateur étape par étape ;
- ▶ s'appuie sur les modules du dossier `utils/`.

À retenir

L'utilisateur standard lance `app.py` après la préparation des fichiers d'entrée.

Modules Python utilisés par l'application : utils/

Module	Rôle
load.py	charger les fichiers d'entrée
clean.py	nettoyer et standardiser les données utiles
mapping.py	appliquer les correspondances de variables
cohort.py	construire la cohorte selon les critères choisis
temporal.py	calculer les variables liées au temps
quality.py	produire les contrôles qualité
export.py	générer les fichiers de sortie
profile.py	lire et valider les profils JSON
display.py	afficher les résultats dans Streamlit
text.py	gérer les textes, libellés et messages

Documentation disponible dans docs/

Fichier	Rôle
overview_github.pdf	notice visuelle du dépôt GitHub
overview_github.tex	source LaTeX modifiable de cette notice
documentation.tex	documentation utilisateur détaillée
guide_execution_sql.md	guide pour lancer les extractions SQL
scenarios.md	scénarios d'utilisation de l'application
scenarios.tex	source LaTeX des scénarios
README.md	sommaire du dossier docs/

Résultats attendus après exécution

Fichiers produits

- ▶ export patient structuré ;
- ▶ rapport qualité ;
- ▶ trace d'exécution.

Utilisation

Ces sorties servent de base au contrôle, à la relecture puis aux analyses de recherche clinique.

- ▶ vérifier la cohorte ;
- ▶ relire les alertes qualité ;
- ▶ exploiter l'export final.

L'utilisateur sait alors quels fichiers ont été utilisés et quel résultat a été généré.

Workflow résumé



1. Exécuter les requêtes du dossier `sql/`.
2. Générer les fichiers avec `scripts/load_final_all.py`.
3. Charger les fichiers CSV/XLSX dans Streamlit ou utiliser `samples/`.
4. Choisir le mapping et le profil dans `conf/`.
5. Lancer les contrôles qualité.
6. Générer l'export final.